Project Charter

Sapientia AI: Intelligent OSHA Compliance and Safety Documentation Platform

Version 2.0 — Expanded Feature Set

Pranjal Sanjay Patil
MS Project Management, Northeastern University

Date: June 24, 2026

This document is an official project charter establishing scope, objectives, technical architecture, timeline, and success metrics for the Sapientia AI platform. It serves as both a development roadmap and a professional work sample demonstrating applied construction technology competency.

1. Project Overview

1.1 Background and Problem Statement

Construction sites generate significant volumes of safety documentation that must comply with OSHA 29 CFR 1904 recordkeeping standards. The OSHA 300 Log, 301 Incident Report, and 300A Annual Summary require accurate, timely, and consistent documentation across all recordable incidents. In practice, project engineers and safety officers spend between two to four hours per incident completing these forms manually, often introducing inconsistencies, omission errors, and delayed filings that expose employers to citations.

Sapientia AI addresses this gap by automating the generation, population, and review of OSHA 300/301 documentation using a large language model backend. Version 1.0 delivered the core automation workflow. Version 2.0, defined in this charter, expands the platform with two critical new modules: an AI-powered Toolbox Talk Generator and a Weekly Safety Summary Report Engine.

1.2 Project Purpose

The purpose of this project is to build a production-ready, demonstrable construction safety technology tool that: (a) solves a real compliance problem faced by general contractors and subcontractors, (b) demonstrates Pranjal Patil's technical capability in Python, AI integration, and construction domain knowledge, and (c) serves as a differentiated work sample during applications to construction technology-forward employers including DPR Construction, Skanska, Kiewit, and BWSC.

1.3 Project Sponsor and Owner

Field	Detail
Project Owner	Pranjal Sanjay Patil
Platform Name	Sapientia AI
Version	2.0
Repository	E:\Sapientia\sapientia-ai
Run Command	streamlit run app.py
Local URL	http://localhost:8501

2. Project Objectives

2.1 Primary Objectives

- Expand Sapientia AI from a single-module OSHA 300/301 tool to a three-module safety documentation platform.
- Deliver a Toolbox Talk Generator that produces trade-specific, incident-type-specific safety briefings in under 10 seconds.
- Deliver a Weekly Safety Summary Report Engine that converts raw daily log uploads into a structured PDF narrative.
- Maintain a Streamlit-based single UI that unifies all three modules under one interface.
- Achieve a demo-ready state suitable for screen recording and GitHub portfolio presentation within six weeks.

2.2 Secondary Objectives

- Document the codebase with clear README files, module descriptions, and a Loom walkthrough video.
- Build reusable prompt templates that can be adapted for employer-specific demo customization.
- Ensure the tool handles edge cases: incomplete inputs, ambiguous incident descriptions, and multi-trade sites.

3. Project Scope

3.1 In Scope

- Module 1 (Existing — Enhancement): OSHA 300/301 Automation. Add input validation, error handling, and UI polish.
- Module 2 (New): Toolbox Talk Generator — inputs: trade type, incident type, severity level; output: structured 10-minute safety talk script.
- Module 3 (New): Weekly Safety Summary Report Engine — inputs: uploaded daily log CSV or PDF; output: downloadable PDF narrative with incident counts, trend flags, and corrective action recommendations.
- Integration layer: Unified Streamlit sidebar navigation connecting all three modules.
- GitHub repository with clean commit history, README, and requirements.txt.

3.2 Out of Scope

- Mobile application development.
- Integration with external OSHA reporting APIs or OSHA RIS system.
- Multi-user authentication or role-based access control.
- Real-time incident tracking or IoT sensor integration.
- Commercial deployment or SaaS productization in this phase.

3.3 Assumptions

- Claude API (claude-sonnet-4-6) remains accessible at current pricing during development.
- Input files will be in standard formats: CSV for logs, PDF for incident reports.
- The tool is deployed and run locally; no cloud hosting is required for portfolio purposes.

4. Technical Architecture

4.1 Technology Stack

Layer	Technology	Purpose
Frontend UI	Streamlit	Single-page app with module navigation
API Backend	FastAPI	REST endpoints for each module
AI Engine	Claude API (LangChain)	Prompt orchestration and generation
PDF Processing	PyMuPDF / ReportLab	Input parsing and output generation
Data Handling	Pandas	CSV log parsing and aggregation
Environment	Python 3.11 + venv	Isolated dependency management

4.2 Module 2 — Toolbox Talk Generator: Logic Flow

Step 1: User selects trade (Concrete, MEP, Steel, Excavation, Roofing, General).

Step 2: User selects incident type (Fall, Struck-by, Caught-in/between, Electrical, Heat, Chemical).

Step 3: User selects severity level (Near-miss, First Aid, Recordable, Lost Time).

Step 4: FastAPI endpoint receives inputs and constructs a structured prompt via LangChain.

Step 5: Claude API generates a 400-600 word toolbox talk script with: opening safety statistic, incident scenario narrative, five corrective behaviors, sign-off attendance format.

Step 6: Streamlit displays output. User can copy text or download as PDF via ReportLab.

4.3 Module 3 — Weekly Safety Summary Report Engine: Logic Flow

Step 1: User uploads daily safety log (CSV with columns: Date, Trade, Incident Type, Description, Corrective Action, Status).

Step 2: Pandas parses the CSV, counts incidents by type, flags open corrective actions, and computes a weekly incident rate.

Step 3: Aggregated data is passed to Claude API with a structured prompt requesting a weekly narrative.

Step 4: Claude generates: executive summary paragraph, incident type breakdown, top three risk areas, recommended focus areas for next week.

Step 5: ReportLab compiles the narrative plus Pandas-generated data tables into a downloadable PDF report.

5. Project Timeline

5.1 Work Breakdown Structure

Phase	Task	Duration	Deliverable
Phase 1	Environment setup + Module 1 polish	Days 1–3	Clean venv, updated requirements.txt, Module 1 UI refresh
Phase 2	Module 2 build — Toolbox Talk Generator	Days 4–10	Working FastAPI endpoint + Streamlit UI + PDF download
Phase 3	Module 2 testing and prompt tuning	Days 11–13	3 test runs per trade/incident combination; prompt v3 locked
Phase 4	Module 3 build — Weekly Report Engine	Days 14–22	CSV parser + Claude narrative + ReportLab PDF output
Phase 5	Module 3 testing with sample data	Days 23–25	5 sample weekly logs processed; output reviewed for accuracy
Phase 6	Integration + unified UI navigation	Days 26–30	Single Streamlit app with sidebar module switcher
Phase 7	Documentation + GitHub + Loom video	Days 31–36	README, demo video (5 min), GitHub repo published
Phase 8	Buffer + final QA	Days 37–42	Edge case testing, UI cleanup, final demo recording

6. Success Metrics and Acceptance Criteria

6.1 Functional Criteria

- Module 2 generates a complete toolbox talk in under 10 seconds for any valid input combination.
- Module 3 produces a formatted PDF report from a standard CSV log upload without manual intervention.
- All three modules are accessible from a single Streamlit interface without restart.
- PDF outputs are formatted, readable, and suitable for direct workplace use.

6.2 Portfolio Criteria

- GitHub repository has a README that explains the tool's purpose, tech stack, and how to run it in under 5 minutes of reading.
- Loom demo video runs 4–6 minutes and shows all three modules end-to-end.
- Code is organized into modular files (not a single monolithic script) with clear naming conventions.

6.3 Interview Readiness Criteria

- Can explain the tool's architecture in 90 seconds without technical jargon.
- Can articulate the real construction site problem it solves with a specific scenario.
- Can demonstrate the tool live during a video interview with a pre-loaded sample dataset.

7. Risk Register

Risk	Likelihood	Impact	Mitigation
Claude API prompt outputs inconsistent toolbox talk quality across trades	Medium	High	Lock prompt template after Phase 3; version-control prompts
CSV log format varies by site; parser breaks on non-standard columns	High	Medium	Build a column mapping UI step; provide a sample CSV template for users
ReportLab PDF output formatting is poor on long narrative text	Medium	Medium	Test with worst-case long outputs; apply fixed max token limits in API calls
Scope creep: adding features beyond the three modules	High	Low	Enforce this charter as the scope document; log future ideas separately
API cost overrun during testing	Low	Low	Use claude-haiku for testing loops; switch to claude-sonnet only for final validation

8. References

Anthropic. (2024). Claude API documentation. Anthropic. https://docs.anthropic.com

LangChain. (2024). LangChain Python documentation. LangChain. https://python.langchain.com/docs

Occupational Safety and Health Administration. (2023). Recordkeeping rule: Recording and reporting occupational injuries and illnesses (29 CFR Part 1904). U.S. Department of Labor. https://www.osha.gov/recordkeeping

Streamlit Inc. (2024). Streamlit documentation. Streamlit. https://docs.streamlit.io

FastAPI. (2024). FastAPI documentation. Tiangolo. https://fastapi.tiangolo.com

---